

Modelo Relacional de la Orden de Trabajo (OT) — AMS

Fecha: 2026-05-12

Owner: David Cabezas

Versión: 1.1

Audiencia: Carlos Iturra, Magdalena Ortega, Jorge Alquinta

1. Cadena principal

La OT vive dentro de una cadena de 3 entidades secuenciales:

Orden	Entidad	PK	Identidad usuario
1	FieldCapture	`capture_id`	(interno, sin código visible)
2	WorkRequest	`request_id`	`AV-NNNNN` (aviso SAP)
3	ManagedWorkOrder	`wo_id`	`OT-2026-NNNNN`

Cada uno con su lifecycle:

Entidad	Estados
FieldCapture	(no tiene estados, solo timestamp)
WorkRequest	DRAFT → APROBADO → LINKED (también RECHAZADO terminal)
ManagedWorkOrder	CREADO → LIBERADO → PLANIFICADO → EN_PROGRAMACION → PROGRAMADO → EN_EJECUCION → CERRADO (también REPROGRAMADO, CANCELADO)

2. Entidades hijas de la OT (apuntan vía `wo_id`)

Tabla	Propósito	Cardinalidad
`audit_log`	Trazabilidad inmutable	1 OT : N entries (típico 15-40)
`backlog_items`	Cola pre-programación	1 OT : 0..1 backlog_item
`preparativos_ot` (SF-662)	Tracking despacho bodega → patio	1 OT : N preparativos (0-10)
`weekly_programs.wo_ids[]`	Programa semanal (JSON list)	M OTs : 1 program
`rca_analyses`	RCA post-cierre	1 OT : 0..1 RCA
`improvement_actions`	CAPA derivada	1 OT : N improvements
Notes / Comments (JSON inline)	Comentarios dentro de OT	1 OT : N notes (inline)
`execution_checklists`	Checklists pre-ejecución	1 OT : N checklists
`work_assignments`	Asignación detallada	1 OT : N assignments
`time_logs`	Horas reales reportadas	1 OT : N time_logs
`equipment_handovers`	Entrega entre turnos	1 OT : N handovers
`post_maintenance_reviews`	Review post-cierre	1 OT : 0..1 review

`variance_alerts`	Alertas plan vs real	1 OT : N alerts
`sap_upload_packages.wo_ids[]`	Export SAP	M OTs : 1 package

3. Campos JSON embebidos dentro de la OT

La tabla `managed_work_orders` no usa joins SQL para sus relaciones internas — guarda los hijos como JSON arrays para tomar snapshot inmutable en cada estado.

Campo	Estructura
`operations`	`[{seq, description, specialty, hours, completion_pct, actual_hours, task_type}]`
`materials`	`[{code, qty_required, reservation_code, qty_consumed, cost_unit}]`
`assigned_workers`	`[{worker_id, name, specialty}]` (FK lógica a `workforce`)
`support_equipment`	`[{tag, name, equipment_type, hours, notes}]` (FK lógica a `support_equipment`)
`documents`	`[{name, url, type}]` — fotos viven acá como URL
`description_history`	`[{text, edited_at, edited_by}]` — append-only
`post_closure_review`	`[outcome, lessons_learned[], recommendations[]]`

Por qué JSON embebido:

- Snapshot inmutable de cada estado (operaciones al programar pueden diferir de al cerrar)
- Menos joins SQL → vistas más rápidas en dashboards
- Schema flexible para campos custom por planta/cliente
- Trade-off: pierde joins/group-by clásicos, se compensa con lookups en Python en endpoints específicos

4. Relaciones FK reales (con ForeignKey constraint)

Tabla origen	Campo	Apunta a
`work_requests.source_capture_id`	FK	`field_captures.capture_id`
`managed_work_orders.absorbed_by_wo_id`	self-ref	`managed_work_orders.wo_id` (OT PM03 absorbe PM01/PM02)
`criticality_assessments.node_id`	FK	`hierarchy_nodes.node_id`
`functional_failures.function_id`	FK	`functions.function_id`
`failure_modes.functional_failure_id`	FK NOT NULL	`functional_failures.failure_id`
`maintenance_tasks.failure_mode_id`	FK	`failure_modes.failure_mode_id`

Pocos FKs explícitos a propósito — el resto son **lógicos** (apuntan por ID sin constraint DB) para tolerar:

- Borrado de captures sin romper la WR
- Borrado de WRs sin romper la OT (la OT puede ser standalone fast-track P1)
- Migración multi-plant sin cascadas

5. Relaciones lógicas (sin FK formal)

Origen	Campo	Apunta a	Para qué
`managed_work_orders.work_request_id`	string nullable	`work_requests.request_id`	Trazabilidad WR → OT
`managed_work_orders.equipment_id`	string	`hierarchy_nodes.node_id` o `.tag`	Equipo físico
`managed_work_orders.technical_location`	string	`hierarchy_nodes` (TL SAP)	Ubicación técnica SAP-PM
`managed_work_orders.contractor_crew_id`	string nullable	`contractor_crews.crew_id`	Cuadrilla externa
`managed_work_orders.assigned_workers[].worker_id`	string (en JSON)	`workforce.worker_id`	Técnicos asignados
`managed_work_orders.support_equipment[].tag`	string (en JSON)	`support_equipment.tag`	Grúas, andamios
`managed_work_orders.materials[].code`	string (en JSON)	`inventory_items.material_code`	Repuestos SAP
`managed_work_orders.materials[].reservation_code`	string (en JSON)	(sistema externo SAP)	Reserva BOM en SAP del cliente
`audit_log.entity_id` (con `entity_type`='managed_work_order')	string	`wo_id`	Trazabilidad universal
`preparativos_ot.wo_id` (SF-662)	string indexed	`wo_id`	Tracking despacho bodega → patio
`backlog_items.wo_id`	string indexed	`wo_id`	Cola pre-programación
`weekly_programs.wo_ids[]` (JSON)	list	`wo_id`	Programa semanal
`rca_analyses.wo_id`	string	`wo_id`	RCA post-cierre
`improvement_actions.wo_id`	string nullable	`wo_id`	CAPA derivada de OT
`execution_checklists.wo_id`	string	`wo_id`	Checklists
`work_assignments.wo_id`	string	`wo_id`	Asignación
`time_logs.wo_id`	string	`wo_id`	Horas reales
`variance_alerts.wo_id`	string	`wo_id`	Alertas plan vs real
`post_maintenance_reviews.wo_id`	string	`wo_id`	Post-cierre

6. Flujo de vida típico

#	Actor	Acción	Resultado
1	Técnico (móvil)	Foto + audio + GPS en patio	FieldCapture creado
2	Sistema	POST /capture/	WorkRequest DRAFT, código AV-NNNNN
3	Supervisor	PUT /work-requests/{id}/validate {action: APPROVE}	WR.status = APROBADO + auto-add backlog

4	Planner	POST /managed-work-orders/from-wr	ManagedWorkOrder CREADO + WR.status = LINKED
5	Planner	Define operations + materials + workers	JSON arrays poblados
6	Planner	/release → /plan → /schedule (drag-drop Horarios)	Cada transición = audit_log entry
7	Técnico	/start → /progress (HH parcial)	actual_start + operations[i].actual_hours
8	Supervisor	/close con signature + gate_acks	status=CERRADO + audit_log final
9	Engineer (opcional)	RCA + improvement_actions + lesson_learned RAG	Tablas relacionadas pobladas

7. Decisiones de diseño relevantes

7.1 JSON embebido vs tablas separadas

Aspecto	JSON embebido (elegido)	Tabla separada
Snapshot inmutable	Sí, gratis	No, requiere historizar
Schema flexible	Sí	No, migración
Joins SQL	No	Sí
Performance dashboard	Sí	Más lento
Integridad referencial	Manual ('/orphans')	Auto FK

7.2 Por qué `work_request_id` NO tiene FK

- **Standalone fast-track:** OTs P1/P2 pueden crearse sin WR previo
- **Robustez ante migraciones:** borrado de WR no rompe OTs ya producidas
- **Compatibilidad SAP:** en SAP el aviso y la orden son entidades semi-independientes

7.3 Concurrency control

- Campo `version` en `managed_work_orders` (SF-562) — optimistic locking
- Frontend manda `If-Match: <version>` en PUT
- Backend retorna 409 Conflict si la versión cambió
- Excepciones: `support_equipment` y `materials.reservation_code` bypass (metadata accesoria — Jorge 2026-04-28)

7.4 Soft-delete vs hard-delete

- OTs NO se borran físicamente. Estados terminales: `CERRADO` , `CANCELADO`
- Audit log: append-only (SF-660 política inmutabilidad)
- Solo el script `scripts/purge_audit_log_5y.py --confirm` puede eliminar audit > 5 años

7.5 Soporte multi-plant

- `plant_id` en `managed_work_orders` (string, no FK formal contra `plants`)
- IDOR-scoping en todos los endpoints: si el usuario no es admin/manager, su `plant_id` JWT gatea acceso
- Tests `test_security_*` cubren los escenarios cross-plant

8. Cardinalidades típicas

Relación	Cardinalidad
FieldCapture → WR	1 : 1 (cada captura genera 1 WR)
WR → OT	1 : 0..1 (máximo 1 OT, o ninguna si rechazada)
OT → operations	1 : N (típico 5-15 por OT)
OT → materials	1 : N (0-50 según tipo)
OT → assigned_workers	1 : N (típico 1-5 técnicos)
OT → audit_log entries	1 : N (típico 15-40 lifecycle completo)
OT → preparativos	1 : N (0-10 por OT crítica)
OT PM03 → OTs absorbidas	1 : N self-ref

9. Tipos de OT y su impacto

`wo_type`	Origen	Skip planning?	Pre-close gates
PM01 — Correctivo	WR de falla	No	Todas las gates
PM02 — Preventivo	Plan matriz	No	Todas
PM03 — Mejora	Improvement action	No	Todas + absorbe PM01/PM02
Fast-track P1	Imprevisto urgente	Sí — va directo a PROGRAMADO	Todas + supervisor_qa obligatorio
Fast-track P2	Programa en ejecución	Sí	Todas

10. Tablas universales (no específicas de OT)

Tabla	Propósito	Conecta a OT vía
`audit_log`	Trazabilidad inmutable de todo el sistema	`entity_type`='managed_work_order' + `entity_id`=wo_id`
`field_captures`	Capturas de campo (foto + audio + GPS)	Vía WR via `source_capture_id`
`hierarchy_nodes`	Estructura jerárquica del activo (Plant→Area→System→Equipment)	`equipment_id` lógico
`workforce`	Personal técnico	`assigned_workers[].worker_id` lógico
`inventory_items`	Catálogo materiales SAP	`materials[].code` lógico
`support_equipment`	Grúas + andamios + herramientas	`support_equipment[].tag` lógico

11. Cómo mitigamos la pérdida de integridad referencial

Sin FK constraints automáticas, suplimos con:

Mitigación	Detalle
Endpoint <code>/managed-work-orders/orphans`</code> (SF-657)	Detecta inconsistencias activamente (PROGRAMADO sin workers, CERRADO sin <code>actual_end</code> , etc.)
Pre-close gates SF-570	Antes de cerrar OT validan que ops + HH + materiales sean consistentes
Audit log immutable SF-660	Toda transición queda registrada, podemos reconstruir estado
Tests pytest	212 tests en <code>`test_api/`</code> cubren happy path + edge cases
Periodic data audit scripts	<code>`scripts/seed_*</code> validan integridad post-importe

12. Referencias cruzadas

Doc	Path
Database blueprint completo	[docs/DATABASE_BLUEPRINT.pdf](DATABASE_BLUEPRINT.pdf)
Project blueprint sistema	[docs/PROJECT_BLUEPRINT.pdf](PROJECT_BLUEPRINT.pdf)
Audit log policy	[docs/AUDIT_LOG_POLICY.pdf](AUDIT_LOG_POLICY.pdf)
OT-Calendar sync	[docs/OT_CALENDAR_SYNC.pdf](OT_CALENDAR_SYNC.pdf)
Modelo Python	<code>`api/database/models.py`</code> líneas 456 (WR), 534 (OT)
Service layer	<code>`api/services/managed_wo_service.py`</code>
Tests	<code>`tests/test_api/test_managed_wo.py`</code> , <code>`tests/test_api/test_ot_calendar_sync.py`</code>

Generado en commit: [c5911a6](#) · post fix-tests + SF-661 v0.2